

CAN THO UNIVERSITY

COLLEGE OF INFORMATION AND COMMUNICATION TECHNOLOGY



PROJECT – BASIC TOPICS

MAJOR: SOFTWARE ENGINEERING - HIGH-QUALITY PROGRAM

COURSE CLASS: CT239H – 01

WORDLE

Advisor:

Ph.D Phan Phuong Lan

Group member:

B2203560 – Vo Tran Vu Khoa

2nd Semester, Academic Year 2024-2025

ABSTRACT

This project aims to recreate the popular word-guessing game Wordle as a web-based application using the ASP.NET MVC framework. The main objective is to provide users with an interactive and engaging platform to enhance their vocabulary and word recognition skills through a simple yet challenging language puzzle.

The game was developed using a combination of server-side logic in C# and client-side enhancements with JavaScript and AJAX for a smoother user experience. Words are fetched via third-party APIs to store and randomly generated from an internal database. The interface dynamically responds to player inputs, providing visual feedback on each guess, and maintaining user progress through sessions.

The project successfully implemented the full Wordle game logic, including input validation, letter-position matching, and persistent score tracking. Additional features such as audio, animations, and a responsive design were integrated to enhance user engagement.

Overall, the project demonstrates the effective use of ASP.NET MVC for building modern interactive web applications. It highlights the power of combining backend data handling with frontend responsiveness to deliver a seamless game experience. This implementation also serves as a foundation for future expansions such as user authentication, leaderboards, and multiplayer features.

TABLE OF CONTENTS

Introduction	6
Problem Statement.....	6
Objectives	6
Theoretical Foundation.....	7
Application Results.....	10
Software Requirement Specification.....	10
Software Design	24
Testing	48
Conclusion	55
References	56

TABLE OF FIGURES

Figure 1 Application architecture	24
Figure 2 Application Data Structure.....	26
Figure 3 Game play testing flow	48
Figure 4 Point calculation testing flow.....	49
Figure 5 Automatic word adding testing flow.....	50
Figure 6 Manual word adding testing flow	51

TABLE OF ABBREVIATION

Abbreviation	Full form
MVC	Model-View-Controller
AJAX	Asynchronous JavaScript and XML
API	Application Programming Interface
UI	User Interface
RNG	Random number generator
SRNG	Subtractive random number generator

CONTENT

INTRODUCTION

Problem Statement

In today's digital learning environment, interactive and gamified tools have become essential for enhancing user engagement and knowledge retention—especially in the context of language learning. Games that promote vocabulary development not only make the learning process more enjoyable but also improve cognitive skills such as memory and pattern recognition. However, many existing word games are either locked behind paywalls, lack educational value, or are not designed with accessibility and language learning in mind.

While Wordle has gained popularity as a word-guessing game, there remains a gap in web-based, customizable versions that support real-time feedback, user tracking, and the ability to easily integrate new word databases. Additionally, learners often lack access to lightweight, responsive platforms that simulate Wordle mechanics while offering an intuitive interface and the potential for future enhancements.

The problem this project aims to address is the absence of an affordable, interactive, and educational tool that combines vocabulary training with engaging gameplay. Without such a platform, learners and casual players are limited to rigid or non-customizable applications that do not fully support personalized language development or playful learning experiences.

Objectives

The main objective of this project is to design and implement a Wordle-style game using ASP.NET MVC framework. Specifically, we aim to:

- Build an interactive user interface that mimics the mechanics of the original Wordle game.
 - Enable dynamic word validation and result feedback.
 - Integrate features such as word tracking, scoring, and session-based player progress.
 - Ensure that the application runs efficiently across different devices and browsers.
- This project was completed within a set academic semester timeframe.

THEORETICAL FOUNDATION

The project is developed using ASP.NET MVC, a powerful web application framework that follows the Model-View-Controller architectural pattern. The system interacts with a backend SQL Server database to manage user data and game states, while client-side interactions are enhanced using JavaScript, AJAX, and Bootstrap for responsive UI design.

ASP.NET MVC Framework

ASP.NET MVC is used to structure the application logically by separating concerns. The Model handles data and logic, the View manages the user interface, and the Controller processes user inputs and updates the model or view accordingly. This framework facilitates maintainability and scalability.

C# and .NET 6.0

C# serves as the primary programming language for both server-side logic and data manipulation. The .NET 6.0 runtime provides the necessary libraries and runtime environment for robust application development.

SQL Server

SQL Server is used as the relational database system to store user accounts, game history, and a repository of valid English words. Entity Framework is employed to manage database operations through a Code First approach.

JavaScript and AJAX

JavaScript enhances user interactivity, enabling features such as real-time word validation and dynamic updates without full page reloads. AJAX is used for asynchronous communication between client and server.

Bootstrap 5

Bootstrap provides a mobile-first, responsive design system that ensures the application is accessible across different devices and screen sizes.

External APIs

Some external APIs like Vercel and Free dictionary are used to provide word information and translation

Random Number Generator

Random number generator (RNG)^[1] is used to generate a random word for each round of the game. .NET framework use a Subtractive random number generator^{[2][3]}

(SRNG) for their System.Random package. The pseudo-code for their algorithm can be described as below:

Initializing SeedArray:

```
mj = MSEED - |seed|;  
seedArray[55] = mj;  
mk = 1 ;
```

Assigning value to the SeedArray:

```
for i = 1 to 54:  
    ii = (21 × i) mod 55;  
    seedArray[ii] = mk;  
    mk = mj - mk;  
    if (mk < 0): mk += MBIG;  
    mj = seedArray[ii] ;
```

Increasing the randomness by looping 4 times:

```
for k = 0 to 3:  
    for i = 1 to 55:  
        seedArray[i] = seedArray[i] - seedArray[1 + (i + 30) mod 55];  
        if seedArray[i] < 0:  
            seedArray[i] += MBIG ;
```

Generate the random number:

```
inext = (inext + 1) mod 56;  
inextp = (inextp + 1) mod 56;  
  
 $X_n = (\text{seedArray}[\text{inext}] - \text{seedArray}[\text{inextp}]) \bmod \text{MBIG};$   
  
If (  $X_n < 0$ :  $X_n += \text{MBIG}$ )  
  
 $\text{seedArray}[\text{inext}] = X_n ;$ 
```

Setting bound to the number:

```
int Next(int max) => (int)((Next() / (double)MBIG) * max);
```


Parameter Description:

- MBIG = 2,147,483,647 (the largest value of int)
- MSEED = 161803398 (“nearly golden ratio” $\times 10^8$)
- MZ = 0 (zero by default)
- SeedArray: A 56 element array of Integer

APPLICATION RESULTS

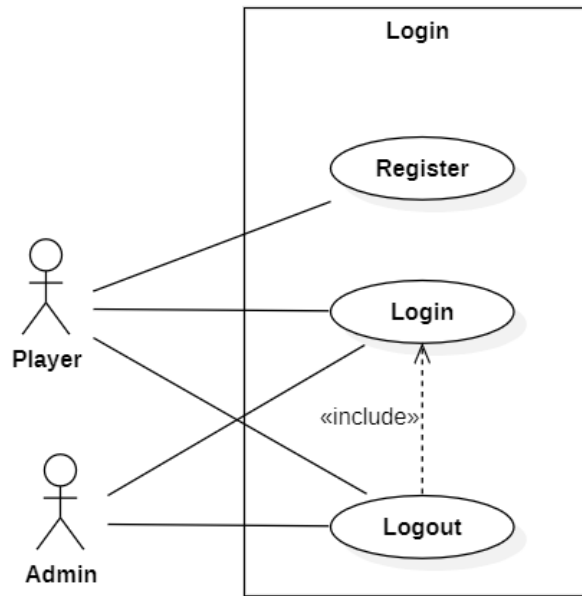
Software Requirement Specification

Overall Description

The system is a web-based Wordle game developed using the ASP.NET MVC framework. It allows users to guess 5-letter English words within a limited number of attempts. The game includes features such as real-time validation of words, visual feedback for correct and incorrect letters, and a scoring mechanism. The application is accessible via modern web browsers and does not require any additional plugins

Functional Requirements

Login

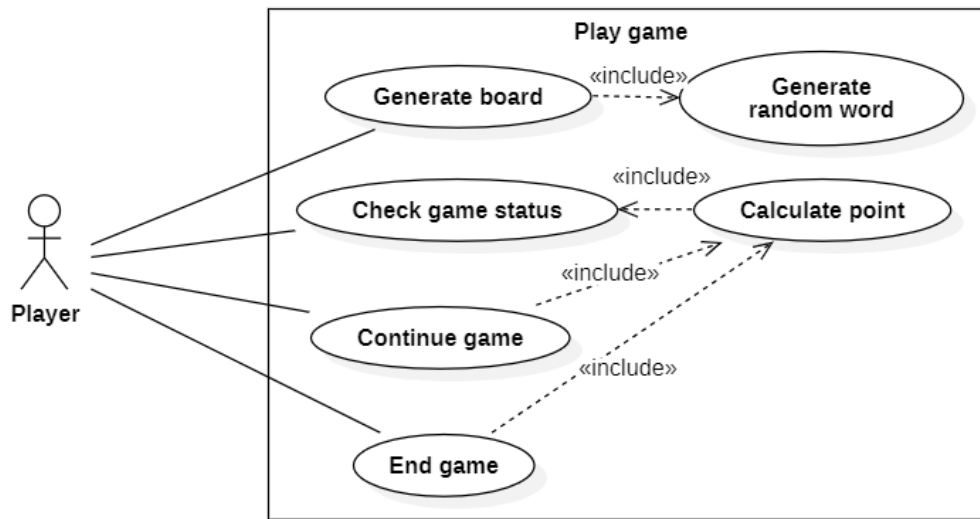


Use Case: Login	ID: UC001
Main actor: User	Priority: Medium
Brief description: <i>The User log in via their account</i>	
Trigger: User	Type: External
Relationship: + Association: User– Login + Include: None + Extend: None + Generalization: None	
Normal flow: 1. The user click on login button 2. The system display a login form 3. The user enter their username and password and click login 4. The system then check if the user has existed 5. The system then display the interface correspond to the user role	
Exceptional flows: 1. User not existed: The system inform the user that there's an error in their input	

Use Case: Logout	ID: UC002
Main actor: User	Priority: Medium
Brief description: The User logout of their account	
Trigger: User	Type: External
Relationship: + Association: User– Logout + Include: Login + Extend: None + Generalization: None	
Normal flow: 1. The user click on logout button 2. The system then clear the user session and return to homepage	
Exceptional flows: None	

Use Case: Register	ID: UC003
Main actor: User	Priority: Medium
Brief description: The User register an account	
Trigger: User	Type: External
Relationship: + Association: User– Register + Include: None + Extend: None + Generalization: None	
Normal flow: 1. The user click on register button 2. The system display a register form 3. The user enter their username and password and click register 4. The system then check if the user has existed 5. The system then check if there's an error in the user's input 6. The system then add a new user to the database	
Exceptional flows: 1. User existed: The system inform the user that there's already an account eisted by that username 2. Invalid input: The system inform the user that there's an error in their input	

Play game



Use Case: Generate board	ID: UC004
Main actor: Player	Priority: High
Brief description: <i>The system generate the playing board</i>	
Trigger: Player	Type: External
Relationship: + Association: Player - Generate board + Include: Generate random word + Extend: None	
Normal flow: 1. The player click on play button 2. The system display a difficulty selection menu 3. The player then select a difficulty 4. The system then generate a random word from the database 5. The system then generate the board base on difficulty selected	
Exceptional flows: None	

Use Case: Generate random word	ID: UC005
Main actor: None	Priority: High
Brief description: <i>The system generate a random word</i>	
Trigger: None	Type: External
Relationship: + Association: None + Include: None + Extend: None + Generalization: None	
Normal flow: 1. Retrieve System Time for Seed Generation. 2. Fetch the total record count from the WoDs table as m. 3. Then a SRNG random number will be calculated as $X = (\text{seedArray}[i] - \text{seedArray}[j]) \bmod(m), \text{ where:}$ • <i>seed</i> is the value obtained from the system time. • seedArray is an array of 56 integer calculated with seed • The result, X, will be in the range from 0 to m-1. 4. The system then sort the WoDs Table and skip X records 5. The system then return the random word (the X + 1 record).	
Exceptional flows: None	

Use Case: Check game status	ID: UC006
Main actor: Player	Priority: High
Brief description: <i>The system check the game status</i>	
Trigger: Player	Type: External
Relationship: + Association: Player - Check game status + Include: None + Extend: None + Generalization: None	

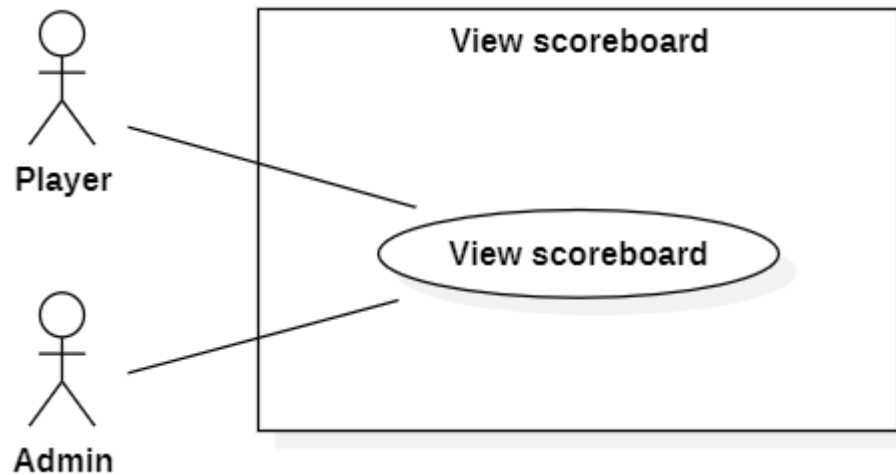
Normal flow: 1. The player enter a word and hit enter 2. The system check if the player has entered the correct word 3. If it is, the system then calculate the point and display the win dialog
Exceptional flows: 1. Is not the correct word and there's still guess chance left: the player can continue guessing 2. Is not the correct word and there's no guess chance left: the system calculate the player's point and display the lose dialog

Use Case: Calculate point	ID: UC007
Main actor: System	Priority: High
Brief description: <i>The system calculate the player's point</i>	
Trigger: System	Type: External
Relationship: + Association: None + Include: Check game status + Extend: None +Generalization: None	
Normal flow: 1. The system check if the game status is won 2. If it is, the point will be calculate as: previous point + (2 + guess left) * multiplier	
Exceptional flows: 1. Game status is lost: the point will be calculate as: prvious point	

Use Case: Continue game	ID: UC008
Main actor: Player	Priority: High
Brief description: <i>The player continue their game after winning a stage</i>	
Trigger: Player	Type: External
Relationship: + Association: Player - Continue game + Include: Calculate point + Extend: None + Generalization: None	
Normal flow: 1. After winning the stage, a win dialog appear prompting the player to continue to next stage or return 2. The player then click continue 3. The system will generate new board	
Exceptional flows: None	

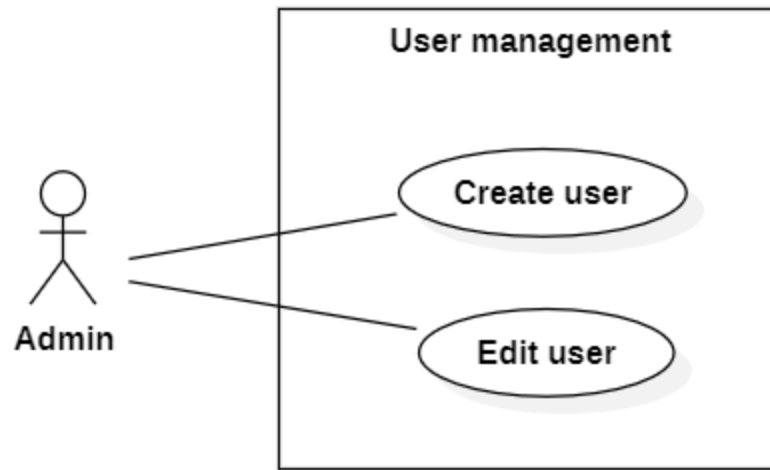
Use Case: End game	ID: UC009
Main actor: Player	Priority: High
Brief description: <i>The player end their game after a round</i>	
Trigger: Player	Type: External
Relationship: + Association: Player - End game + Include: Calculate point + Extend: None + Generalization: None	
Normal flow: 1. After a stage, a dialog appear prompting the player to continue to next round or return 2. The player then click return 3. The system will save the player point to the scoreboard and return to the homepage	
Exceptional flows: None	

View scoreboard



Use Case: View scoreboard	ID: UC010
Main actor: User	Priority: Medium
Brief description: <i>The user view the scoreboard</i>	
Trigger: User	Type: External
Relationship: + Association: User - View scoreboard + Include: None + Extend: None +Generalization: None	
Normal flow: 1. The user click on the view scoreboard button 2. The system query the player's point records and sort it descending by point 3. The system then display the scoreboard	
Exceptional flows: None	

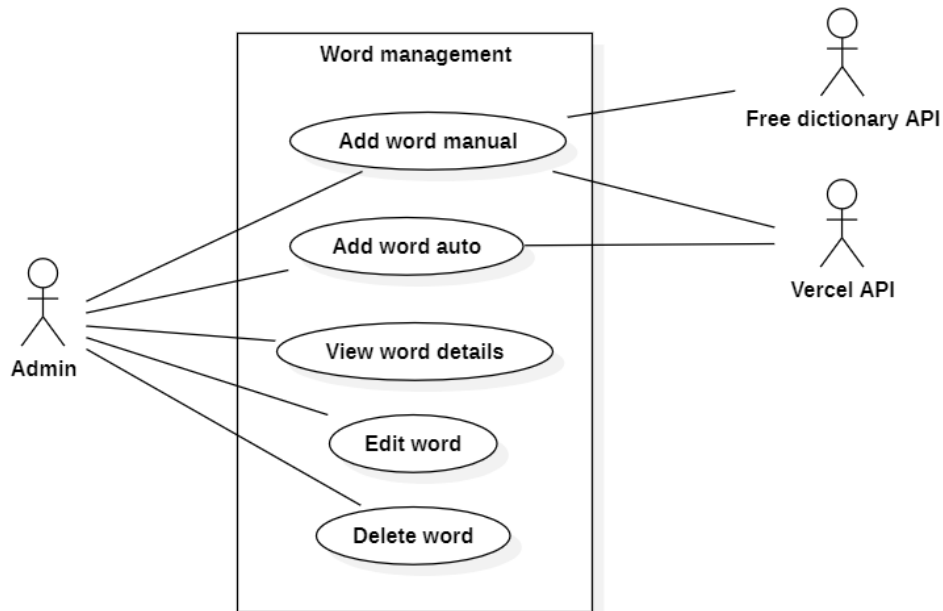
User management



Use Case: Create user	ID: UC011
Main actor: Admin	Priority: Medium
Brief description: <i>The User register an account</i>	
Trigger: Admin	Type: External
Relationship: + Association: Admin– Create user + Include: None + Extend: None +Generalization: None	
Normal flow: 1. The admin click on add user button 2. The system display a add user form 3. The admin enter the username and password and click register 4. The system then check if the user has existed 5. The system then check if there's an error in the admin's input 6. The system then add a new user to the database	
Exceptional flows: 1. User existed: The system inform the admin that there's already an account existed by that username 2. Invalid input: The system inform the admin that there's an error in their input	

Use Case: Edit user	ID: UC012
Main actor: Admin	Priority: Medium
Brief description: <i>The Admin edit a user</i>	
Trigger: Admin	Type: External
Relationship: + Association: Admin–Edit user + Include: None + Extend: None + Generalization: None	
Normal flow: 1. The admin click on edit user button 2. The system display a edit user form 3. The admin enter the user details and click edit 4. The system then check if there's an error in the admin's input 5. The system then update the user in the database	
Exceptional flows: 1. Invalid input: The system inform the admin that there's an error in their input	

Word management



Use Case: Add word manual	ID: UC013
Main actor: Admin	Priority: High
Brief description: <i>The Admin add a new word by hand</i>	
Trigger: Admin	Type: External
Relationship: + Association: Admin– Add word manual + Include: None + Extend: None +Generalization: None	
Normal flow: 1. The admin click on add button and select manual 2. The system the display the manually adding funtion interface 3. The admin enter the word and click fetch 4. The system then fetch the word’s definition and phonetic from the free dictionary api and fill it into the definition and phonetic fields 5. The admin then click add 6. The system then check if the word is existed 7. If not, the system then add the new word into the database	
Exceptional flows: 1. Word existed: The system will inform the admin that there’s already a word like that in the database	

Use Case: Add word auto	ID: UC014
Main actor: Admin	Priority: Medium
Brief description: <i>The Admin use the automatize add word function to add a new word to the database</i>	
Trigger: Admin	Type: External
Relationship: + Association: Admin– Add word auto + Include: None + Extend: None + Generalization: None	
Normal flow: 1. The admin click on add button and select auto 2. The system the display the automatized add funtion interface 3. The admin click play button 4. The system then check if the loop status is “playing” 5. If it is, the system then fetch a word from the vercel random word api 6. The system then check if the word existed 7. If not, the system then fetch the word’s definition and phonetic from the free dictionary api 8. The system then add the new word into the database 9. The system then update the word added table, with the new word that has just been added 10. The system then repeat from step 5	
Exceptional flows: 1. Loop status is not playing: the system will cancel the adding process 2. Word existed: The system will skip the following step and goes back to step 3	

Use Case: View word details	ID: UC015
Main actor: Admin	Priority: Medium
Brief description: <i>The admin view the word details</i>	
Trigger: Admin	Type: External
Relationship: + Association: Admin - View word details + Include: None + Extend: None + Generalization: None	
Normal flow: 1. The admin click on the view word details button 2. The system query the word details 3. The system then display the word details	
Exceptional flows: None	

Use Case: Edit word	ID: UC016
Main actor: Admin	Priority: High
Brief description: <i>The Admin edit a word</i>	
Trigger: Admin	Type: External
Relationship: + Association: Admin–Edit word + Include: None + Extend: None + Generalization: None	
Normal flow: 1. The admin click on edit word button 2. The system display an edit word form 3. The admin edit the word details and click edit 4. The system then check if there’s an error in the admin’s input 5. The system then update the word in the database	
Exceptional flows: 1. Invalid input: The system inform the admin that there’s an error in their input	

Use Case: Delete word	ID: UC017
Main actor: Admin	Priority: Medium
Brief description: <i>The admin delete a word</i>	
Trigger: Admin	Type: External
Relationship: + Association: Admin – Delete word + Include: None + Extend: None + Generalization: None	
Normal flow: 1. The admin click on the delete word button 2. The system then display a confirmation form 3. If the admin click ok, the system will delete the word from the database	
Exceptional flows: 1. The admin click cancel: the system will cancel the process	

Nonfunctional Requirements

- The application must respond to user input within 1 second.
- The UI must be responsive and compatible with mobile and desktop browsers.
- Data must be stored securely using Entity Framework with SQL Server.
- The application must provide a smooth gameplay experience, ensuring response time for all user actions does not exceed 1 second under normal load.
- The interface should be intuitive and accessible for users of all ages. Tooltips and visual hints should be used where necessary.
- The game should maintain consistent behavior and data integrity, even in the event of partial system failure or unexpected input.
- The application should run smoothly on all modern browsers (e.g., Chrome, Firefox, Edge, Safari) and across different operating systems.

Software Design

Application Architecture

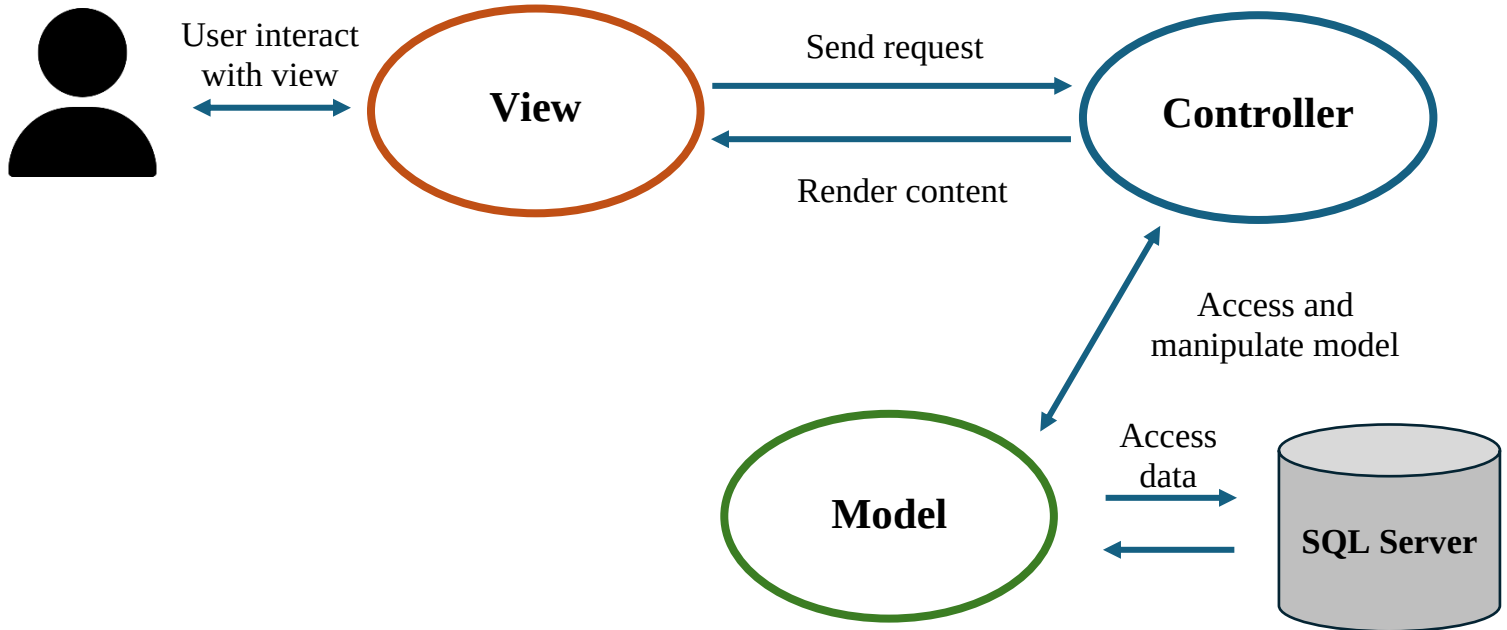


Figure 1 Application architecture

Model

The **Model** represents the data structure and business logic of the application. In this Wordle project, the model includes the *WoDs* class, which encapsulates properties such as:

- **Id:** The unique identifier for each word.
- **Word:** The actual word string.
- **Phonetic:** The phonetic transcription of the word.
- **Definition:** The meaning or definition of the word.

The model also handles interaction with the database using **Entity Framework**, allowing developers to query and manage data using C# objects through the Code First approach.

View

The **View** is responsible for rendering the user interface. Views are created using Razor syntax (.cshtml) and are dynamically generated based on the model's data. Key views in the project include:

- The main game interface, where users guess words.
- The result display, showing correct/incorrect guesses.
- The admin view for adding new words to the database.

Views make use of Bootstrap for styling and JavaScript (with AJAX and fetch APIs) for dynamic updates without full page reloads.

Controller

The **Controller** serves as the intermediary between the Model and the View. It processes user inputs, handles application logic, and decides what data to pass to the view. In this project, the main controller is *WoDsController*, along with other controllers as:

- **WoDsController:** to manage the word (add, edit, delete a word) in the database as well as getting the word details.
- **GameController:** to create the board and fetch a random word from the database for the game.
- **HomeController:** to redirect the user to the homepage.
- **PlayerController:** for the player to manage their account (view, edit details).
- **UserController:** for admin to manage all user account. This controller also process user's login, logout and registering requests.
- **ScoreBoardController:** to redirect the user to the scoreboard and display the player ranking number base on their score records.

Controllers also manage error, and session tracking to enhance gameplay flow.

Data Design

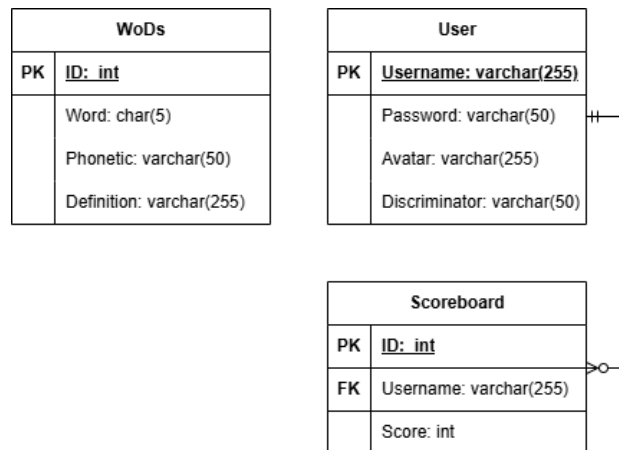


Figure 2 Application Data Structure

Data Dictionary

Word			
Field Name	Data Type	Constraint	Description
ID	int	Primary key	Word's id
Word	Char(5)	Not null	Word name
Phonetic	varchar(50)	Default = "N/a"	Word's phonetic
Definition	Varchar(255)		Word's definition

User			
Field Name	Data Type	Constraint	Description
Username	Varchar(255)	Primary key	User's name
Password	Varchar(50)	Not null	User's password
Avatar	Varchar(255)		User's avatar
Discriminator	Varchar(50)		User's role

Scoreboard			
Field Name	Data Type	Constraint	Description
ID	int	Primary key, identity	Record id
Username	Varchar(255)	Foreign key	Record's user

Score	int		Record score
-------	-----	--	--------------

Detailed Design

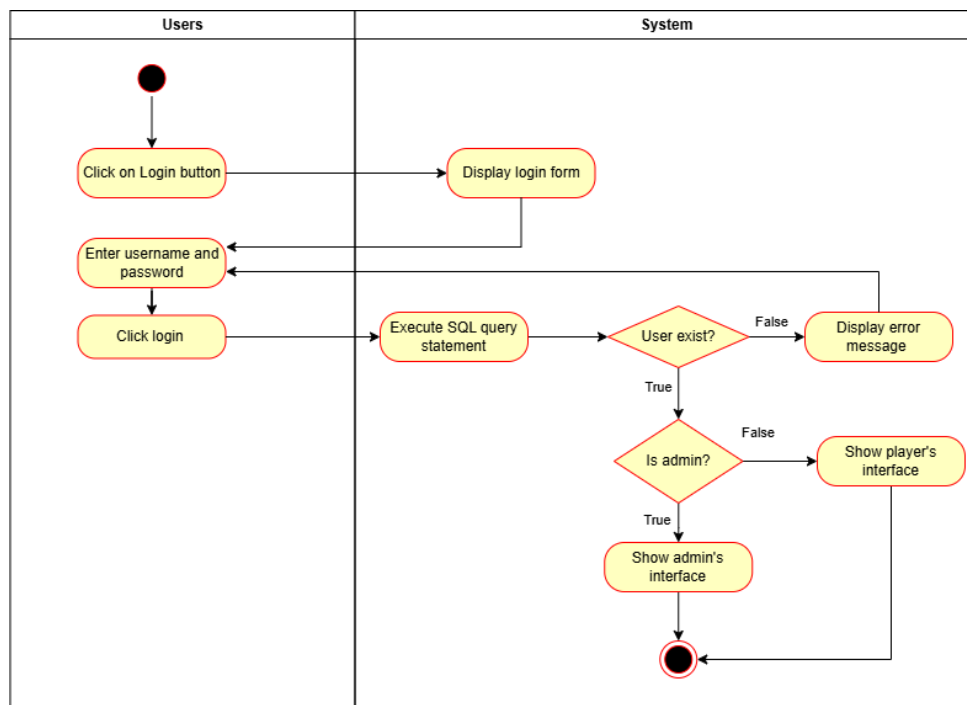
Login

-Login:

The login form is located at the top right of the page, next to 'Home', 'Login', and 'Register' buttons. It contains two input fields: 'Username' (callout 1) and 'Password' (callout 2). Below the password field are two buttons: 'Cancel' (callout 3) and 'Login' (callout 4). There is also an eye icon to toggle password visibility.

No.	Controller Type	Note
1	Text input	Required
2	Text input	Required
3	Button	Return to homepage
4	Button	

No.	Table name	Method			
		Add	Modify	Delete	Query
1	User				X



-Logout:

Home



Username

khoa

Password

12345

Back

Edit

Logout

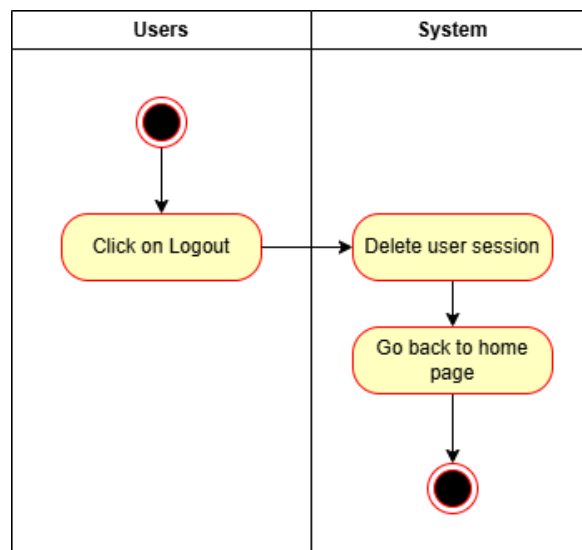
1

2

3

No.	Controller Type	Note
1	Button	Return to homepage
2	Button	
3	Button	

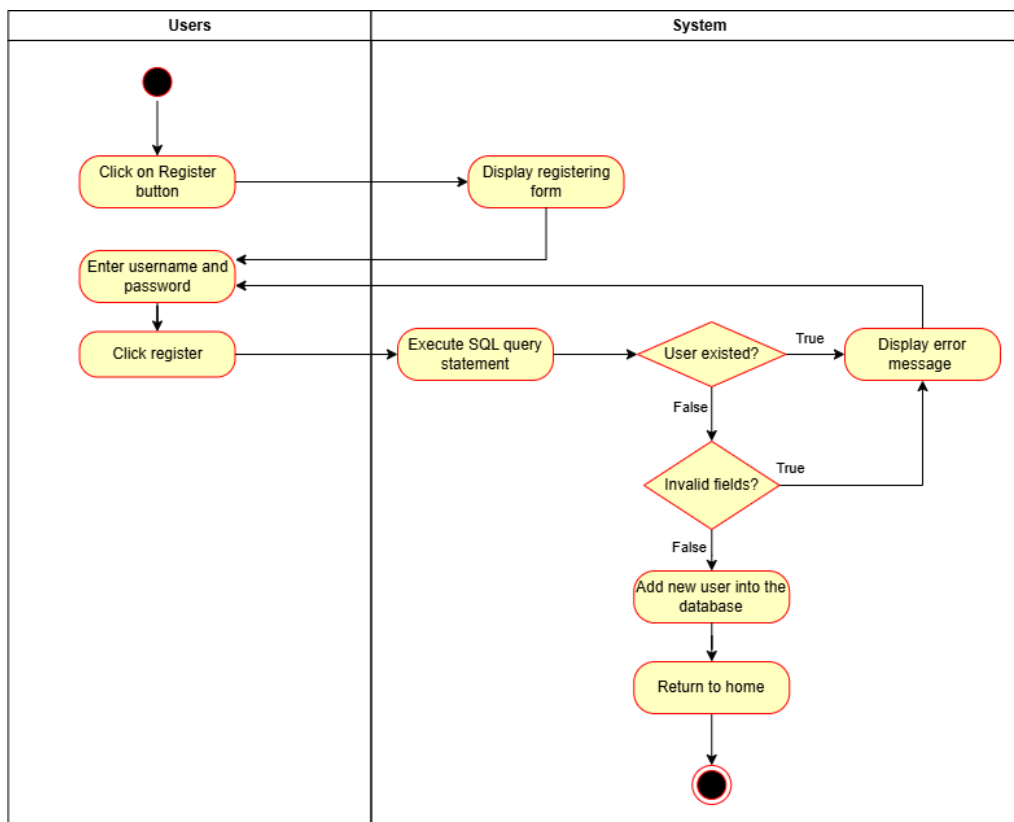
No.	Table name	Method			
		Add	Modify	Delete	Query
1	User				x



-Register:

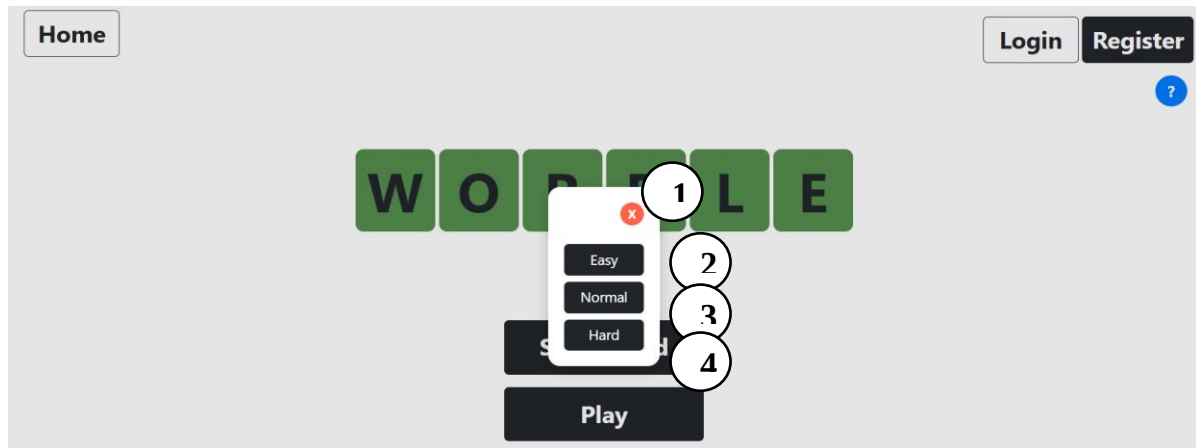
No.	Controller Type	Note
1	Text input	Required
2	Text input	Required
3	Button	Return to homepage
4	Button	

No.	Table name	Method			
		Add	Modify	Delete	Query
1	User	x			



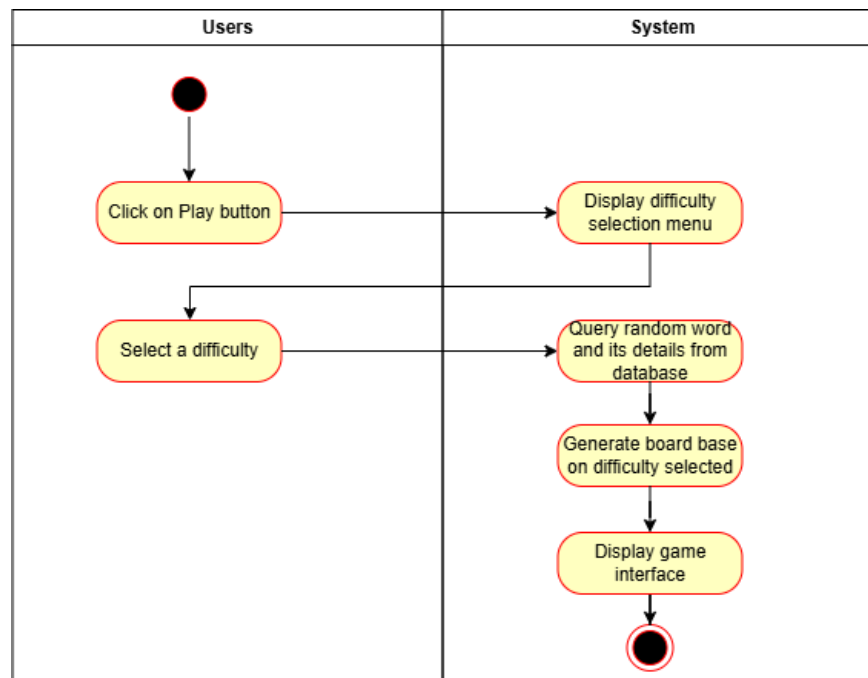
Play game

-Generate board:



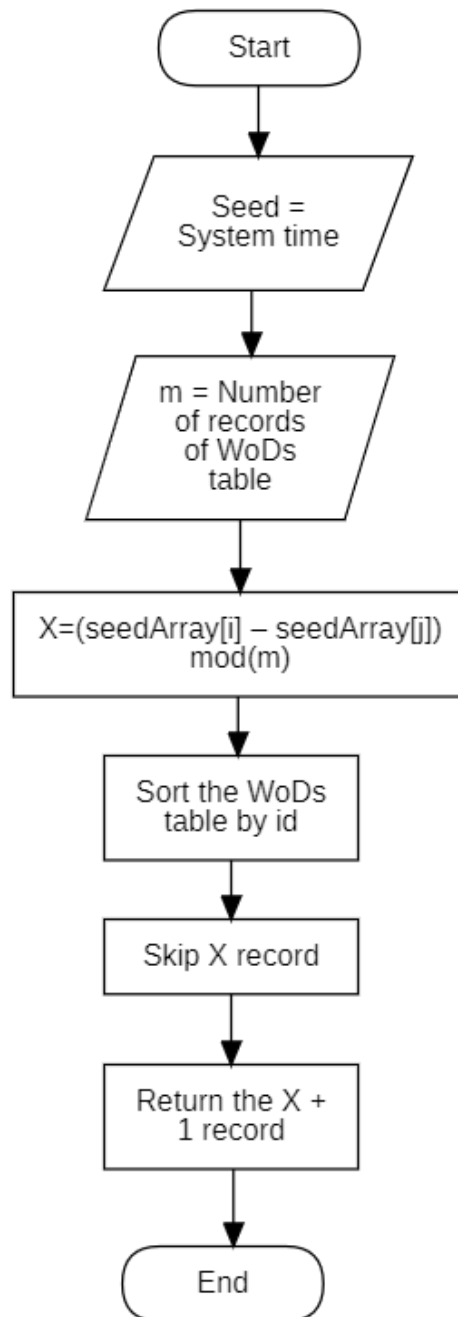
No.	Controller Type	Note
1	Button	Cancel
2	Button	Easy mode
3	Button	Normal mode
4	Button	Hard mode

No.	Table name	Method			
		Add	Modify	Delete	Query
1	WoDs				x

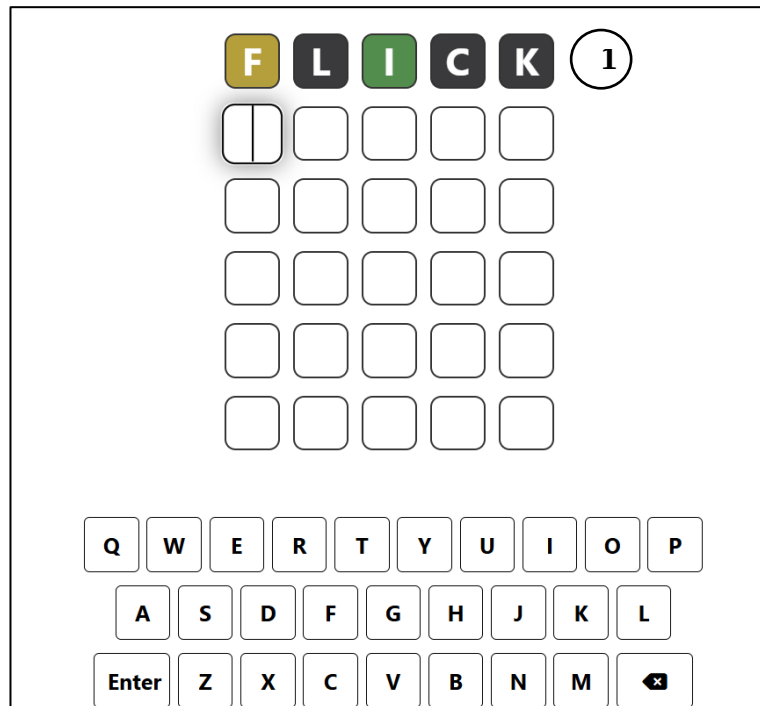


-Generate random word:

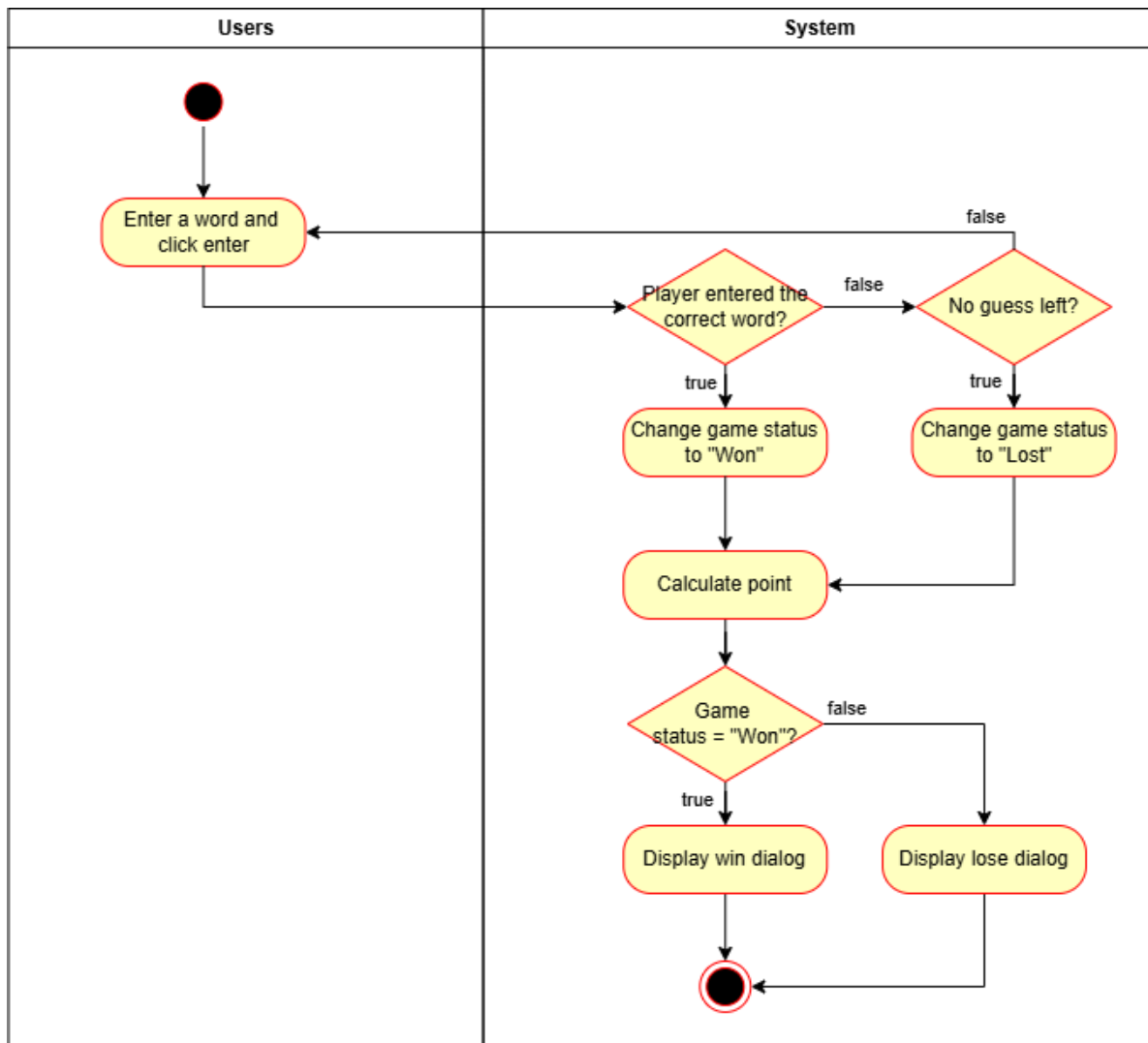
No.	Table name	Method			
		Add	Modify	Delete	Query
1	WoDs				x



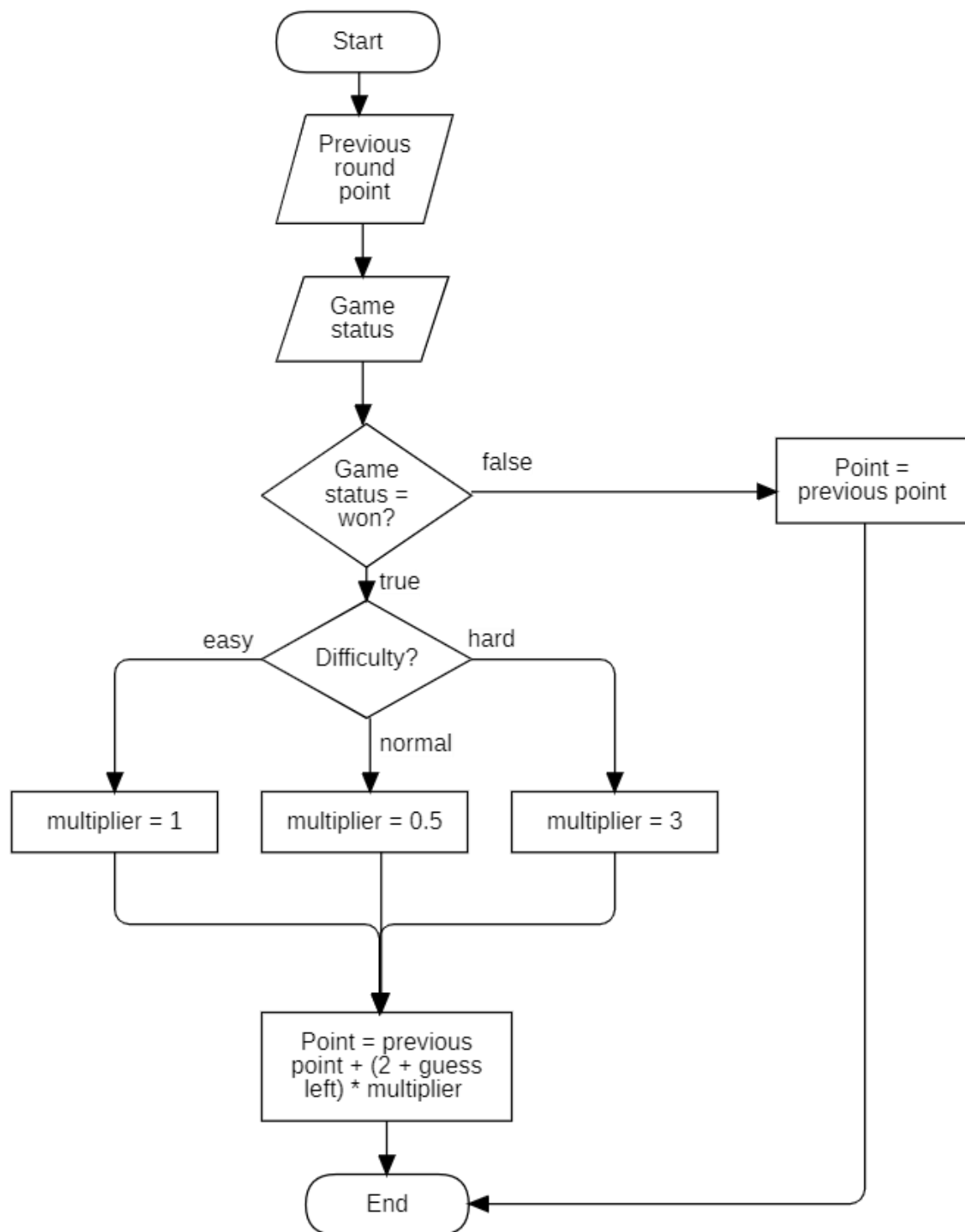
-Check game status:



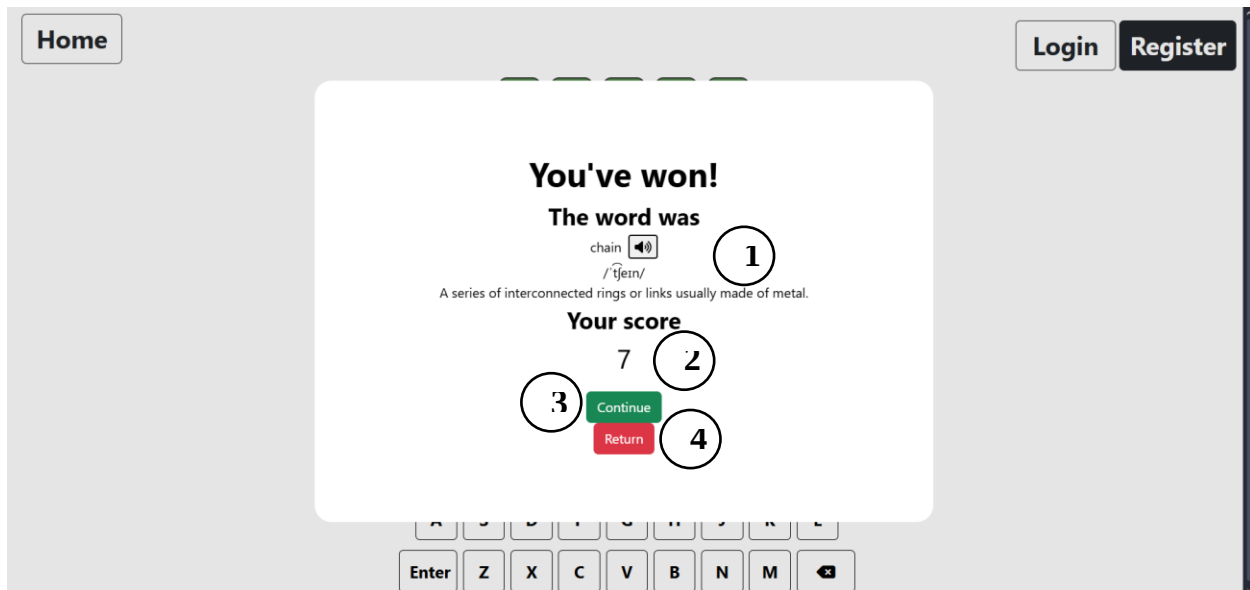
No.	Controller Type	Note
1	Text input	Green = right, Yellow = misplace, Red = wrong



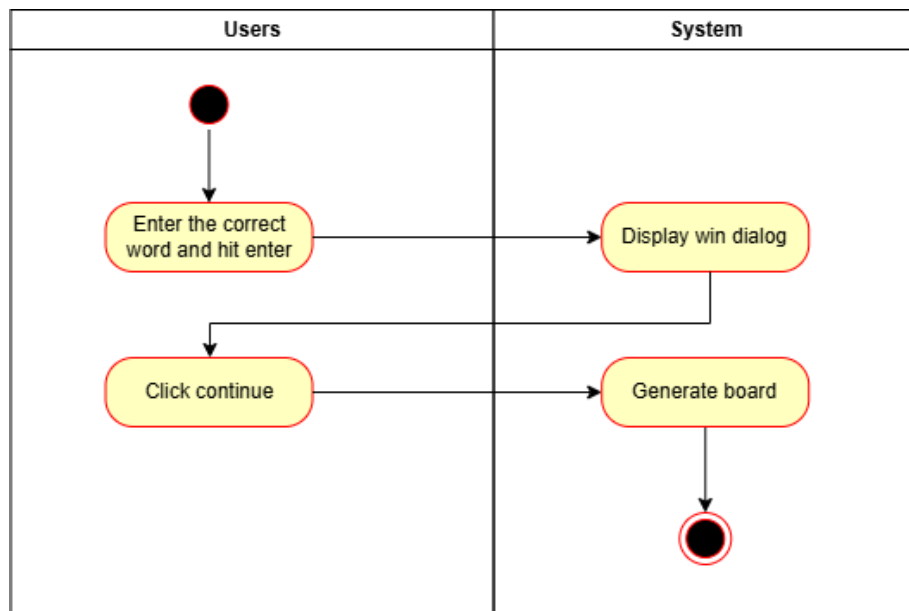
-Calculate point:



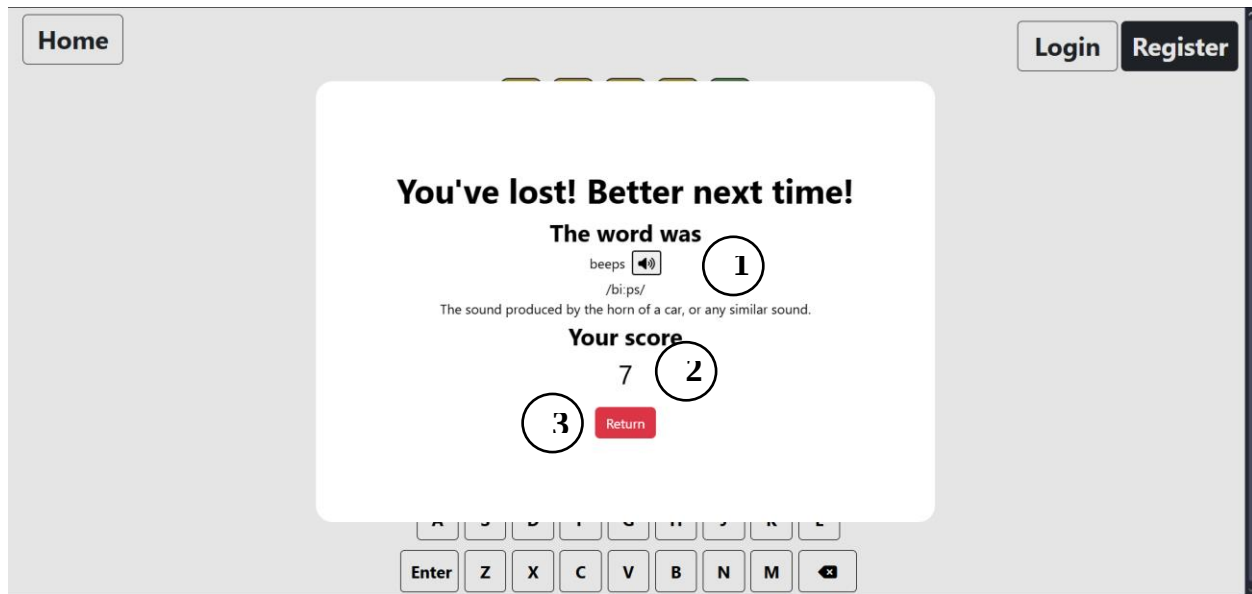
-Continue game:



No.	Controller Type	Note
1	Text	Word details
2	Number	Point
3	Button	
4	Button	

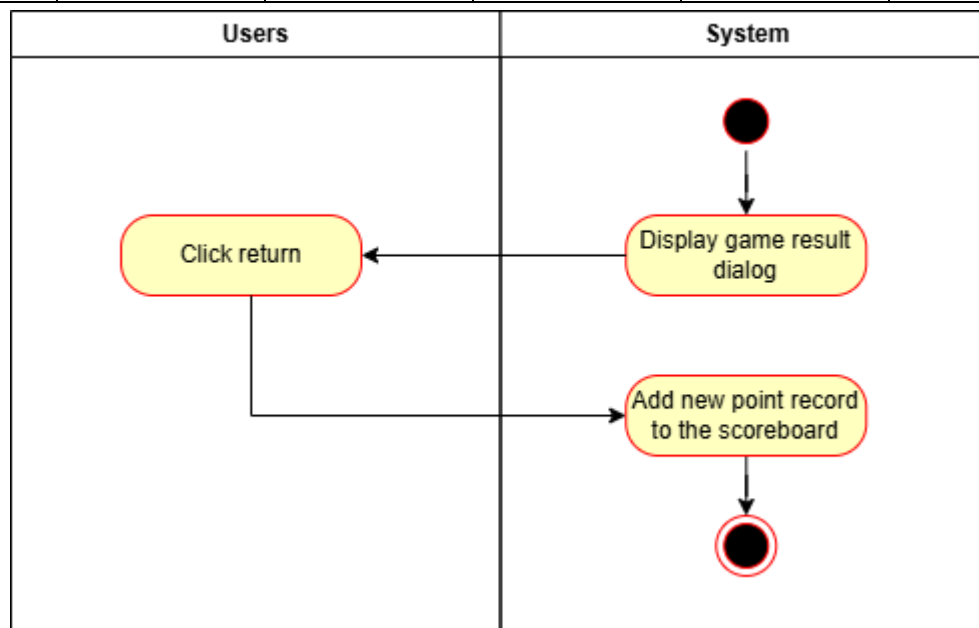


-End game:



No.	Controller Type	Note
1	Text	Word details
2	Number	Point
3	Button	

No.	Table name	Method			
		Add	Modify	Delete	Query
1	User				x



View Scoreboard

Home

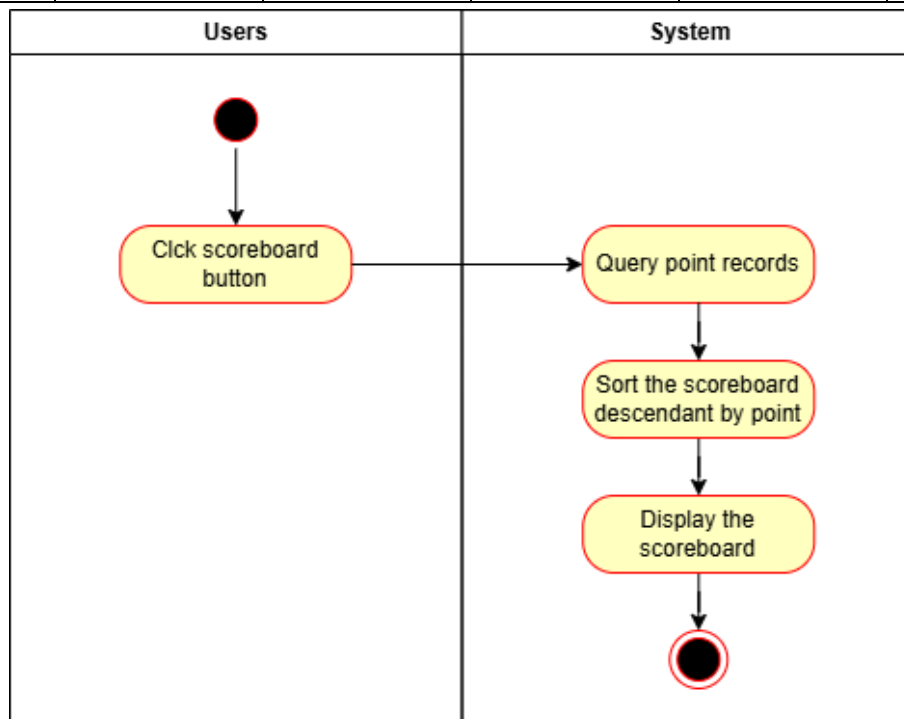
LeaderBoard

Rank	Username	Score
1	khoa	30
2	khoa	28
3	khoa	15
4	khoa	14
5	khoa	8
6	khoa	4

1

No.	Controller Type	Note
1	Text	1 st place = Yellow, 2 nd place = Blue, 3 rd place = red, Others = Gray

No.	Table name	Method			
		Add	Modify	Delete	Query
1	ScoreBoard				x



User management

-Create user:

Home



Create User

Username

1

Password

2

Create

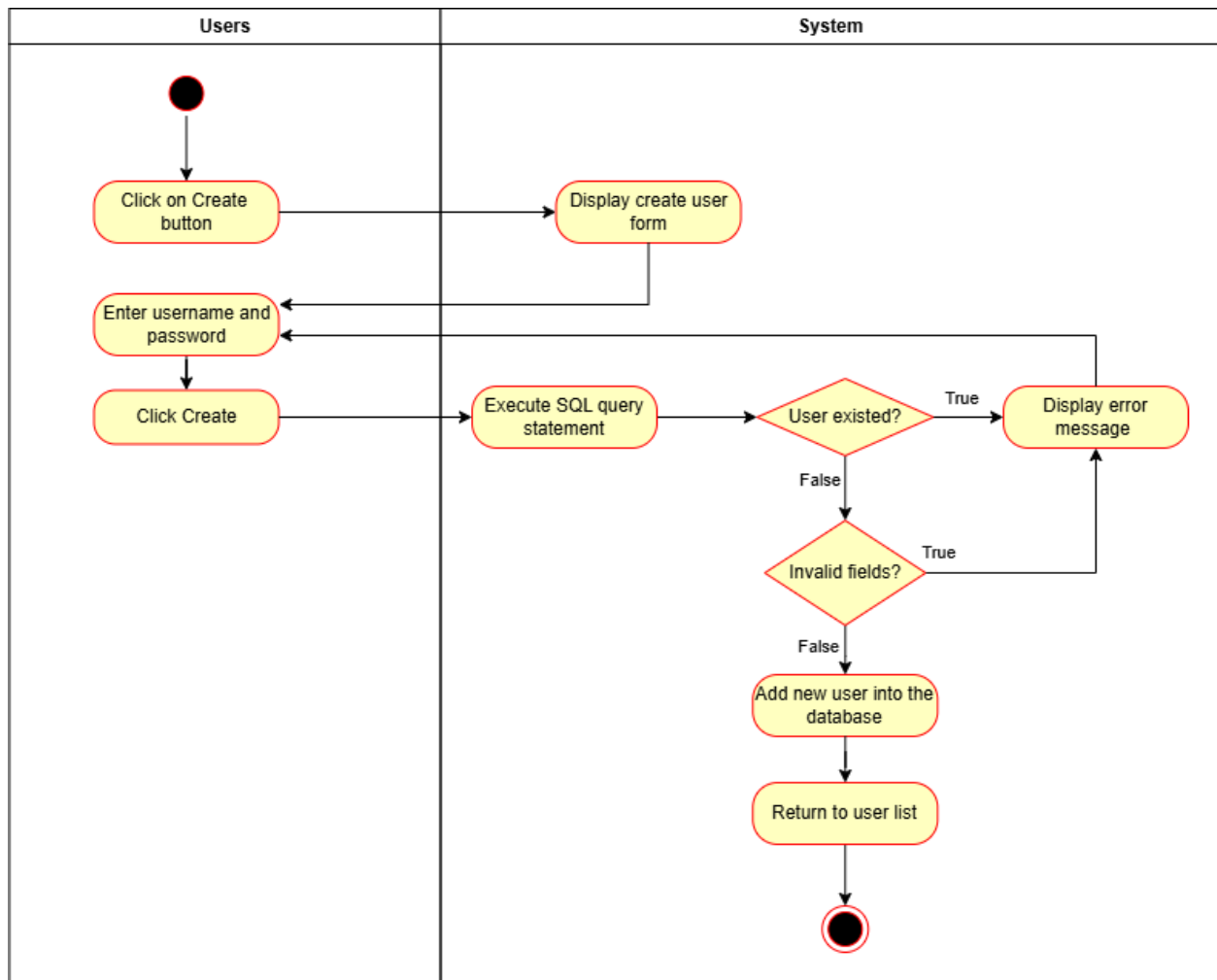
3

Back

4

No.	Controller Type	Note
1	Text input	Required
2	Text input	Required
3	Button	
4	Button	

No.	Table name	Method			
		Add	Modify	Delete	Query
1	Create	x			



-Edit user:

Home

Edit

1

2

3

4

5

Username

Admin

Password

.....

Avatar

Choose File

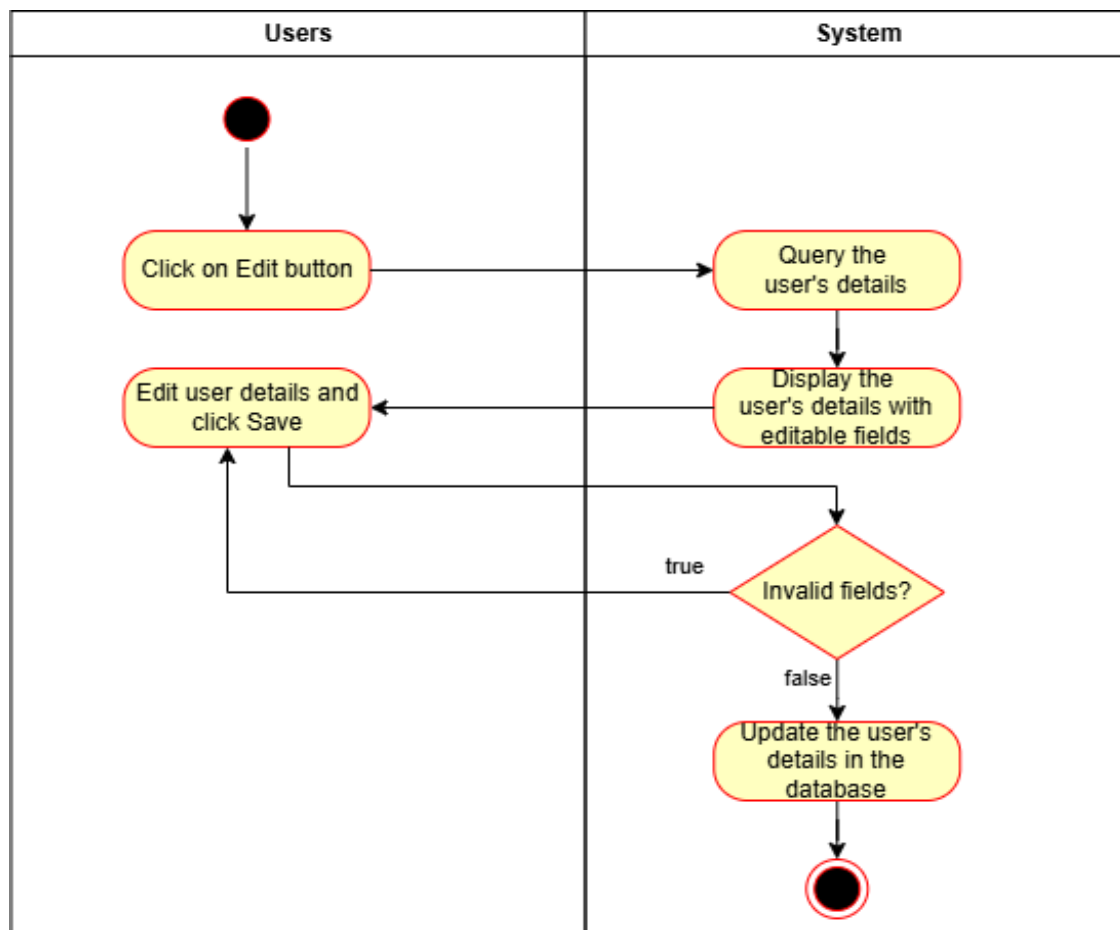
No file chosen

Cancel

Save

No.	Controller Type	Note
1	Text	Required
2	Text input	Required
3	File input	
4	Button	
5	Button	

No.	Table name	Method			
		Add	Modify	Delete	Query
1	User	x			



Word management

-Add word auto:

Home

Back

Total number of word: 755

Number of word added:

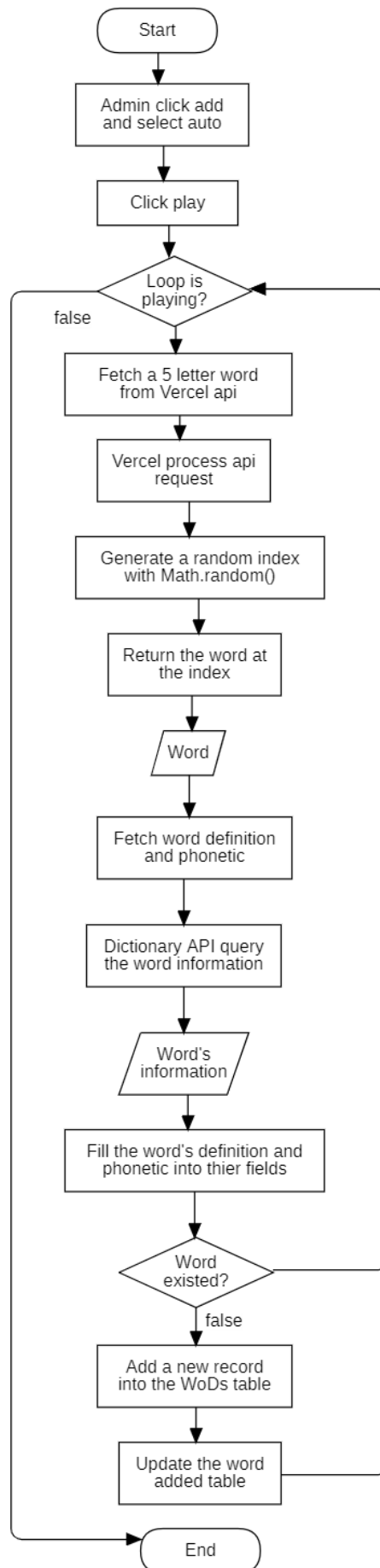
▶

No.

Word

No.	Controller Type	Note
1	Number	
2	Number	
3	Button	
4	Table	

No.	Table name	Method			
		Add	Modify	Delete	Query
1	WoDs	x			

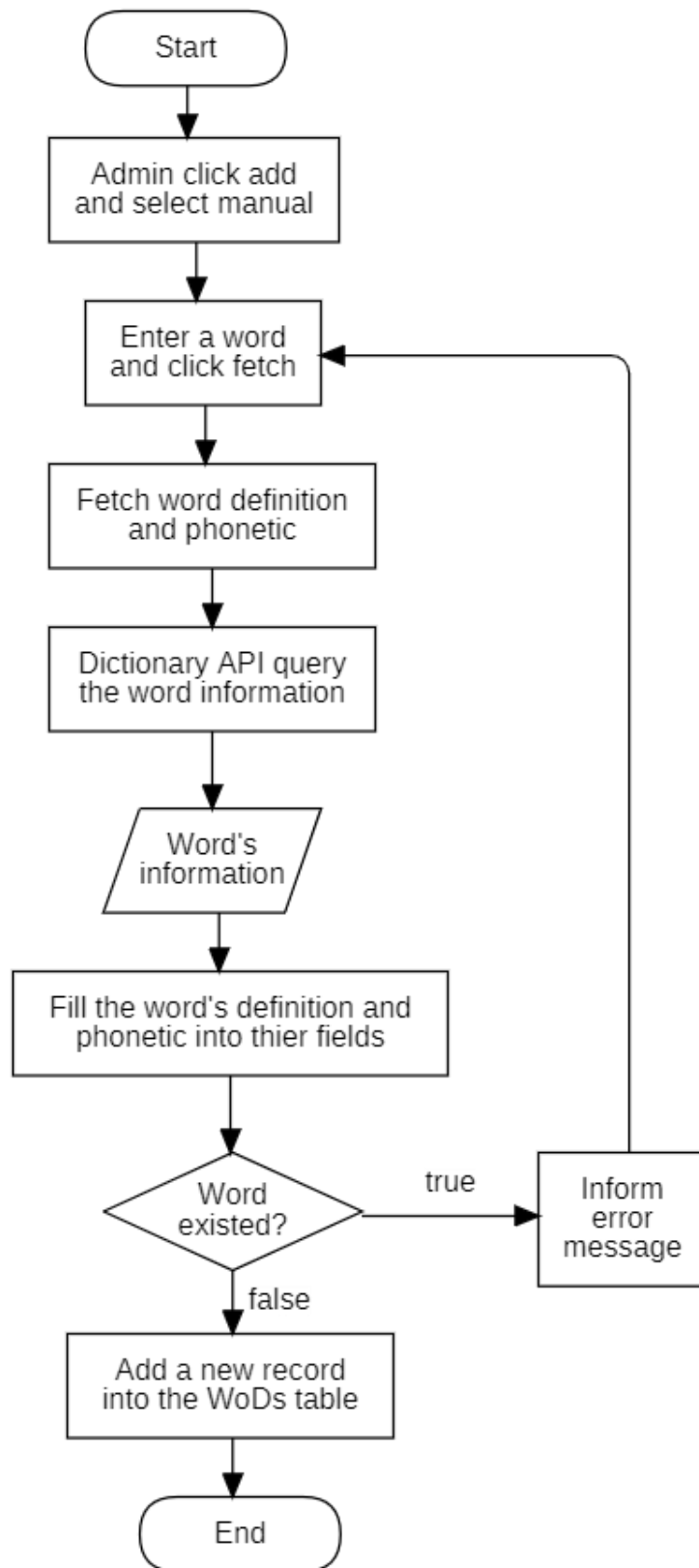


-Add word manual:

The screenshot shows a web application interface for creating a manual entry. At the top left is a 'Home' button. Below it is the title 'CreateManual'. On the right is a user profile icon. The form contains three text input fields labeled 'Word', 'Phonetic', and 'Definition'. A blue button with a book icon is positioned to the right of the 'Phonetic' field. At the bottom left are three buttons: 'Fetch' (green), 'Add' (blue), and 'Back' (yellow). Numbered callouts are placed as follows: 1 points to the 'Word' input field; 2 points to the 'Phonetic' input field; 3 points to the 'Definition' input field; 4 points to the 'Fetch' button; 5 points to the 'Add' button; and 6 points to the 'Back' button.

No.	Controller Type	Note
1	Text input	Required
2	Text input	Required
3	Text input	Required
4	Button	
5	Button	
6	Button	

No.	Table name	Method			
		Add	Modify	Delete	Query
1	WoDs	x			



-View word details:

Home

Details

WoD

Word

abide

Phonetic

/əˈbaɪd/

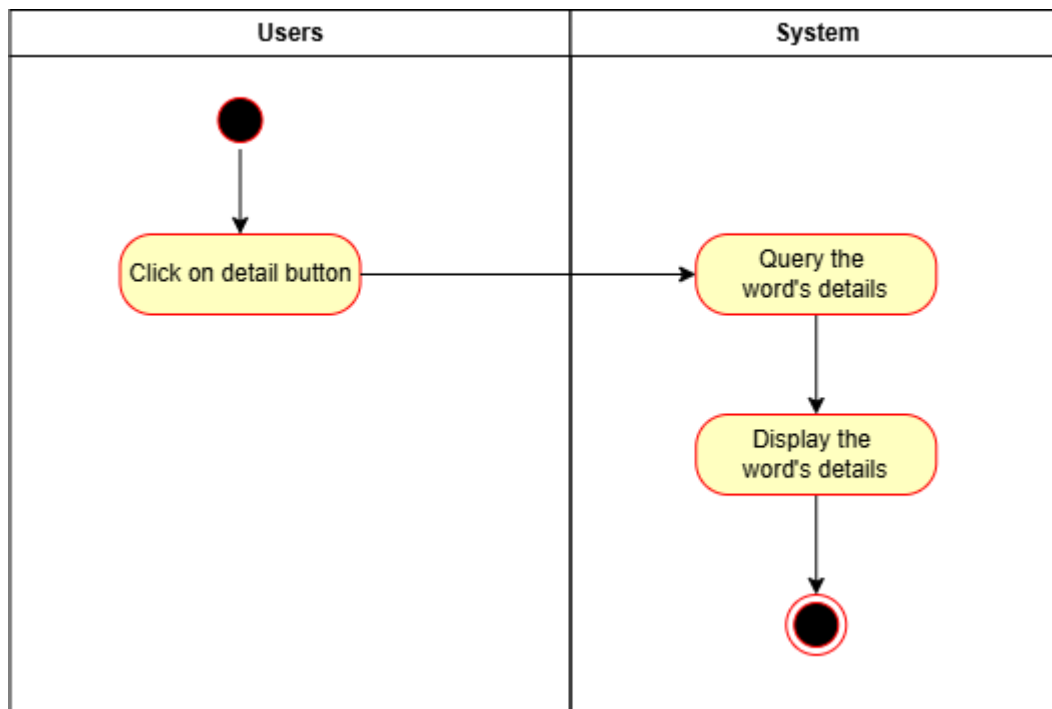
Definition

To endure without yielding; to withstand; await defiantly; to encounter; to persevere.

Back

No.	Controller Type	Note
1	Text	
2	Text	
3	Text	
4	Button	

No.	Table name	Method			
		Add	Modify	Delete	Query
1	WoDs				X



- Edit word:

Home

Edit

WoD

Word

abide

Phonetic

/əˈbaɪd/

Definition

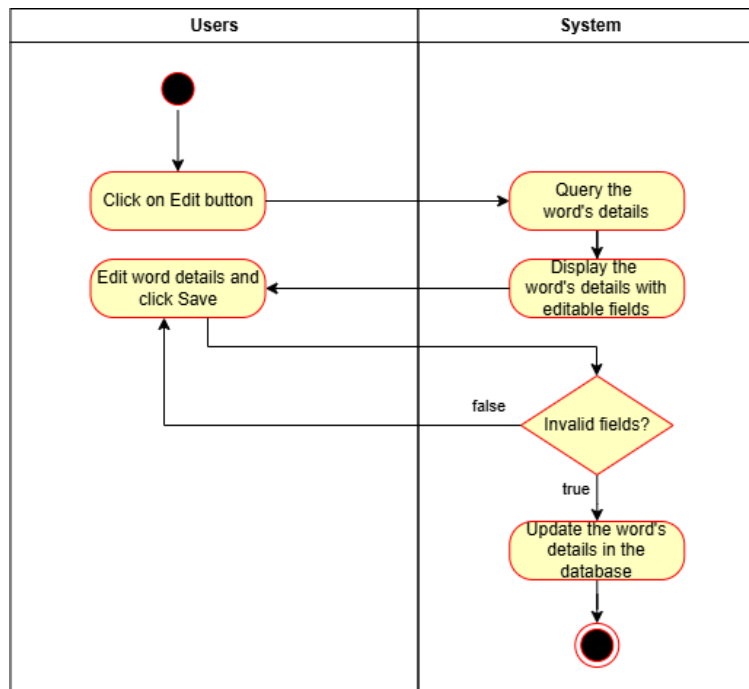
To endure without yielding; to withstand; await defiantly; to encounter; to persevere.

Save

Back

No.	Controller Type	Note
1	Text input	
2	Text input	
3	Text input	
4	Button	
5	Button	

No.	Table name	Method			
		Add	Modify	Delete	Query
1	WoDs		X		X



Delete word:

Home

Delete

Are you sure you want to delete this?

WoD

Word

abide

1

Phonetic

/əˈbaɪd/

2

Definition

To endure without yielding; to withstand; await defiantly; to encounter; to persevere.

3

4

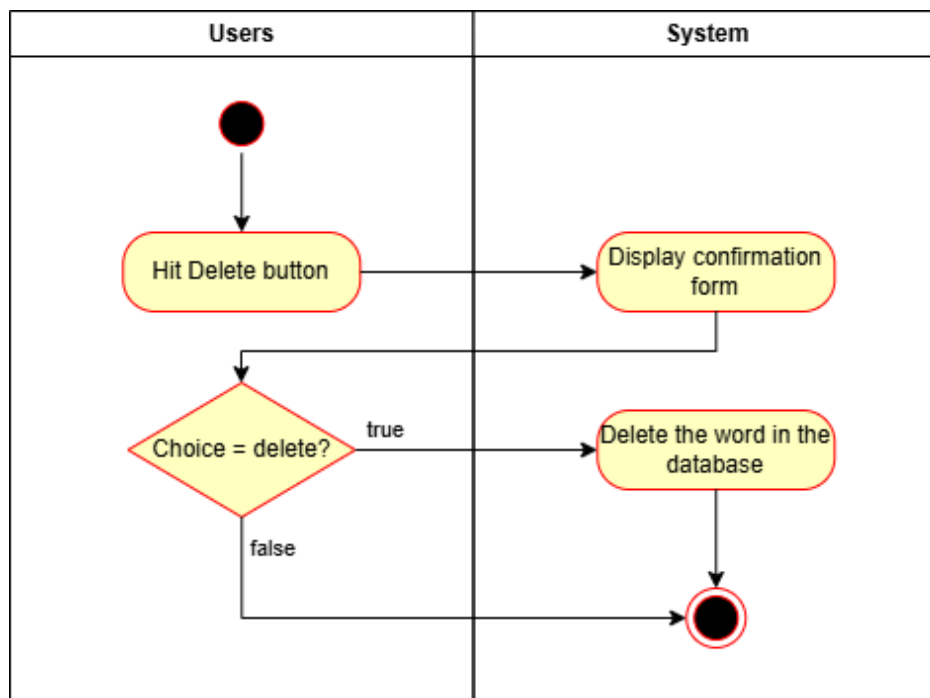
Delete

Back

5

No.	Controller Type	Note
1	Text	
2	Text	
3	Text	
4	Button	
5	Button	

No.	Table name	Method			
		Add	Modify	Delete	Query
1	WoDs			X	



Testing

Test Plan

1. Unit Testing

- Focus: Logic for comparing guessed words, determining color feedback (green/yellow/gray), and random word generation.
- Example Case: Compare guessed word "FLICK" with answer "SNIFF" → Expect result: [Yellow, Gray, Green, Gray, Gray]

2. Integration Testing

- Focus: Communication with the dictionary and random word API.
- Tool: Manual testing.

3. System Testing

- **Focus:**
 - Complete workflow of game play from page load, input guess, win/loss condition.
 - Calculation of player point base on game status and difficulty.
 - Workflow of Admin adding a new word to the game, fetch the word phonetic and definition.
- **Environment:** Local IIS Express.
- **Test Case:**
 - *Gameplay:* Input a 5-letter word → Expect feedback colors and updated UI.

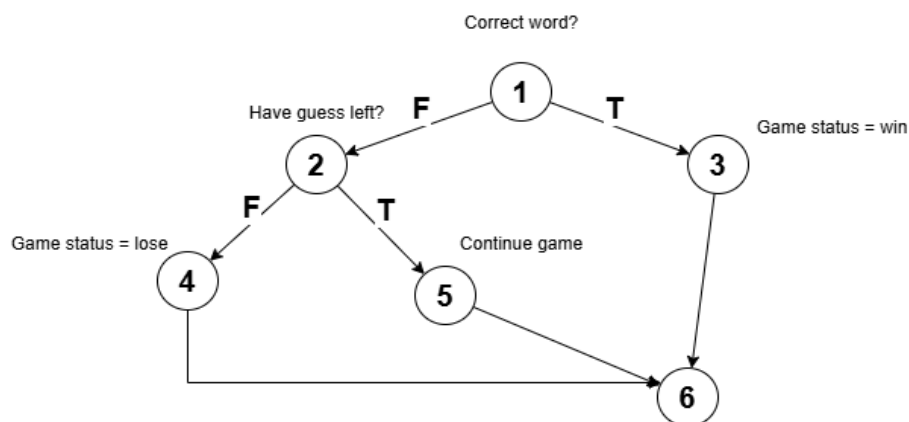


Figure 3 Game play testing flow

TC01: 1 – 3 – 6 (correct word = true)

TC02: 1 – 2 – 5 – 6 (correct word = false, guess left = 1)

TC03: 1 – 2 – 4 – 6 (correct word = false, guess left = 0)

- *Calculate point:* Calculate point by game status (win, loose) and difficulty (easy, normal, hard)

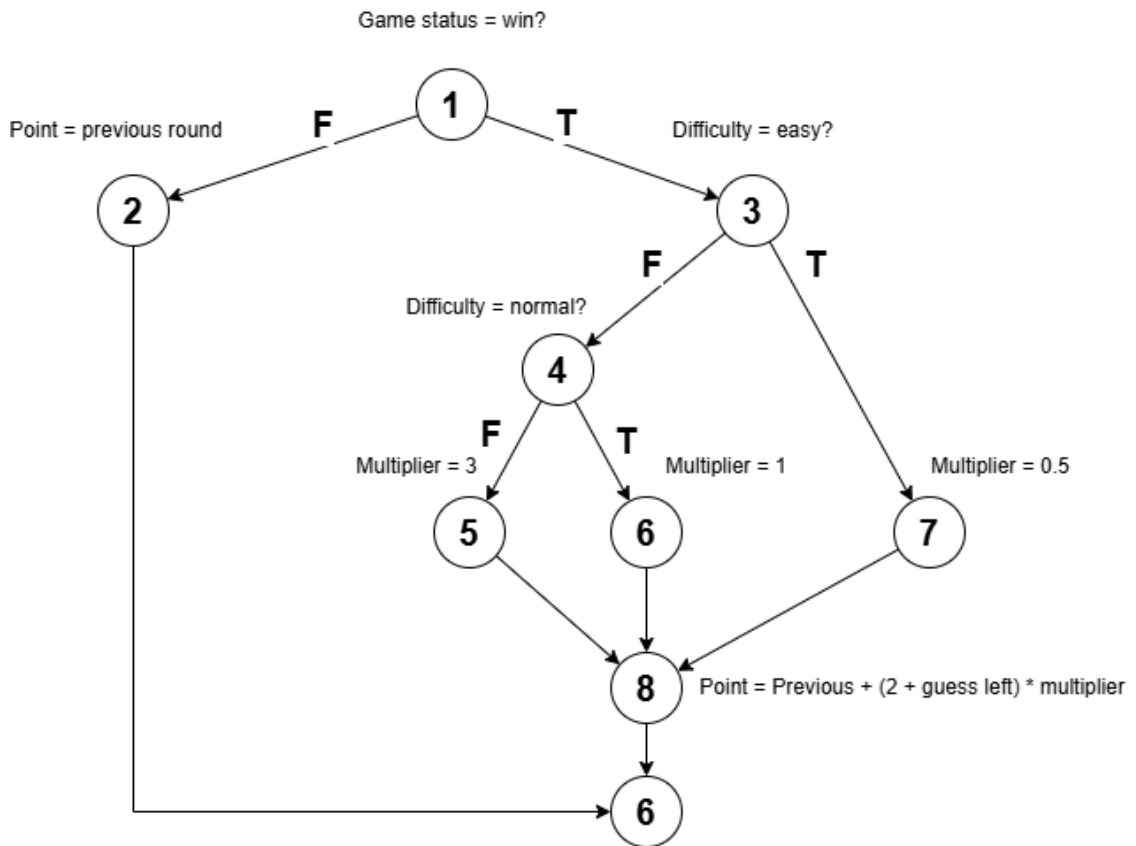


Figure 4 Point calculation testing flow

TC01: 1 – 2 (game status = losse)

TC02: 1 – 3 – 7 – 8 (game status = win, difficulty = easy)

TC03: 1 – 3 – 4 – 6 – 8 (game status = win, difficulty = normal)

TC04: 1 – 3 – 4 – 5 – 8 (game status = win, difficulty = hard)

- *Adding word:*
 - Auto adding:
 - Click play button → System fetch a random word and it meaning → Check if word existed → Add word to database → Update word added table
 - Cancel the adding process by clicking the play button again

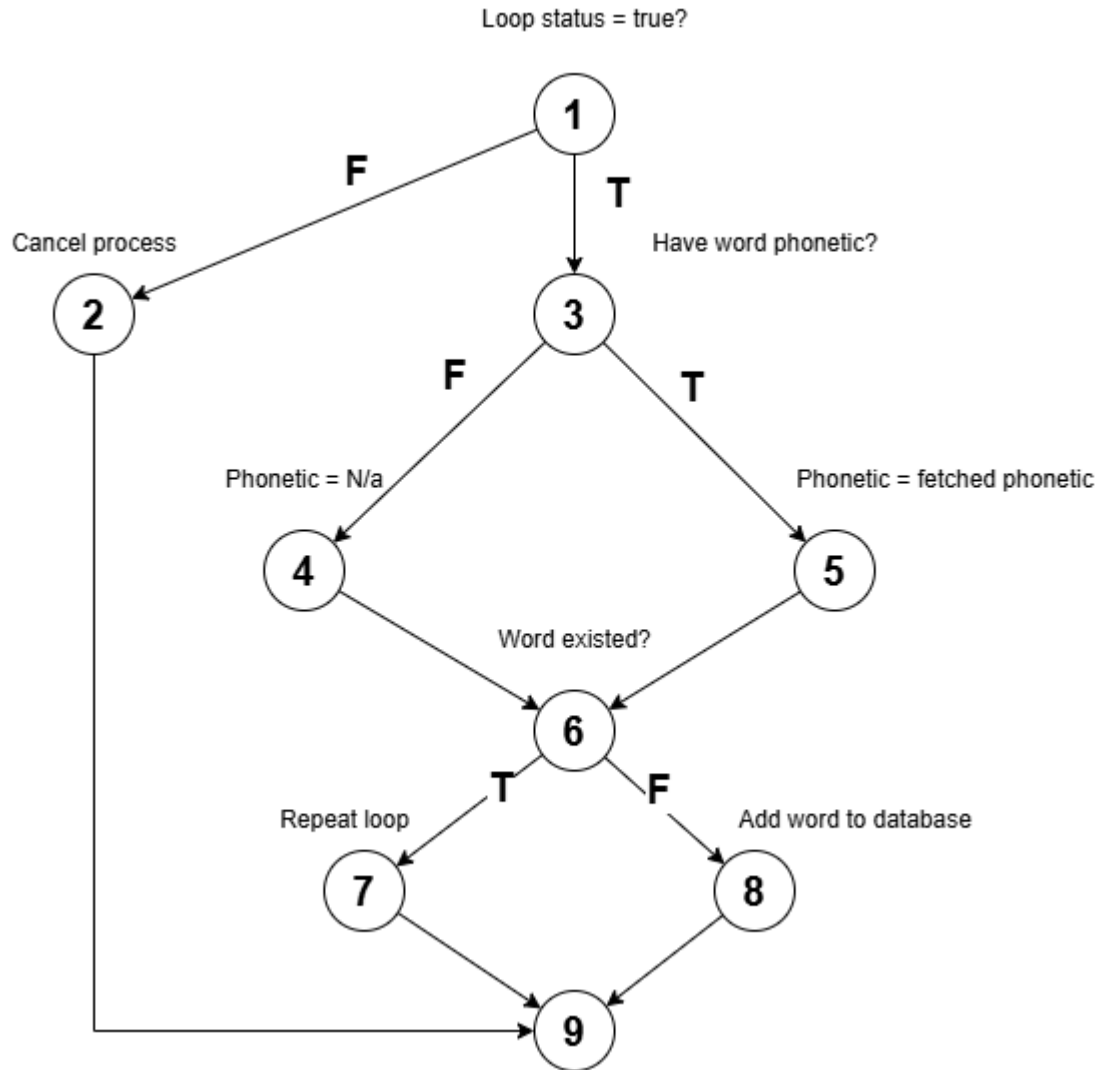


Figure 5 Automatic word adding testing flow

TC01: 1 – 2 (loop status = false)

TC02: 1 – 3 – 4 – 6 – 8 (loop status =true, have no phonetic, word not existed)

TC03: 1 – 3 – 5 – 6 – 8 (loop status =true, have phonetic, word not existed)

TC04: 1 – 3 – 4 – 6 – 7 (loop status =true, have phonetic, word existed)

- Manual adding: Admin enter a word → Fetch word phonetic and definition → Check if word existed → Add word to database

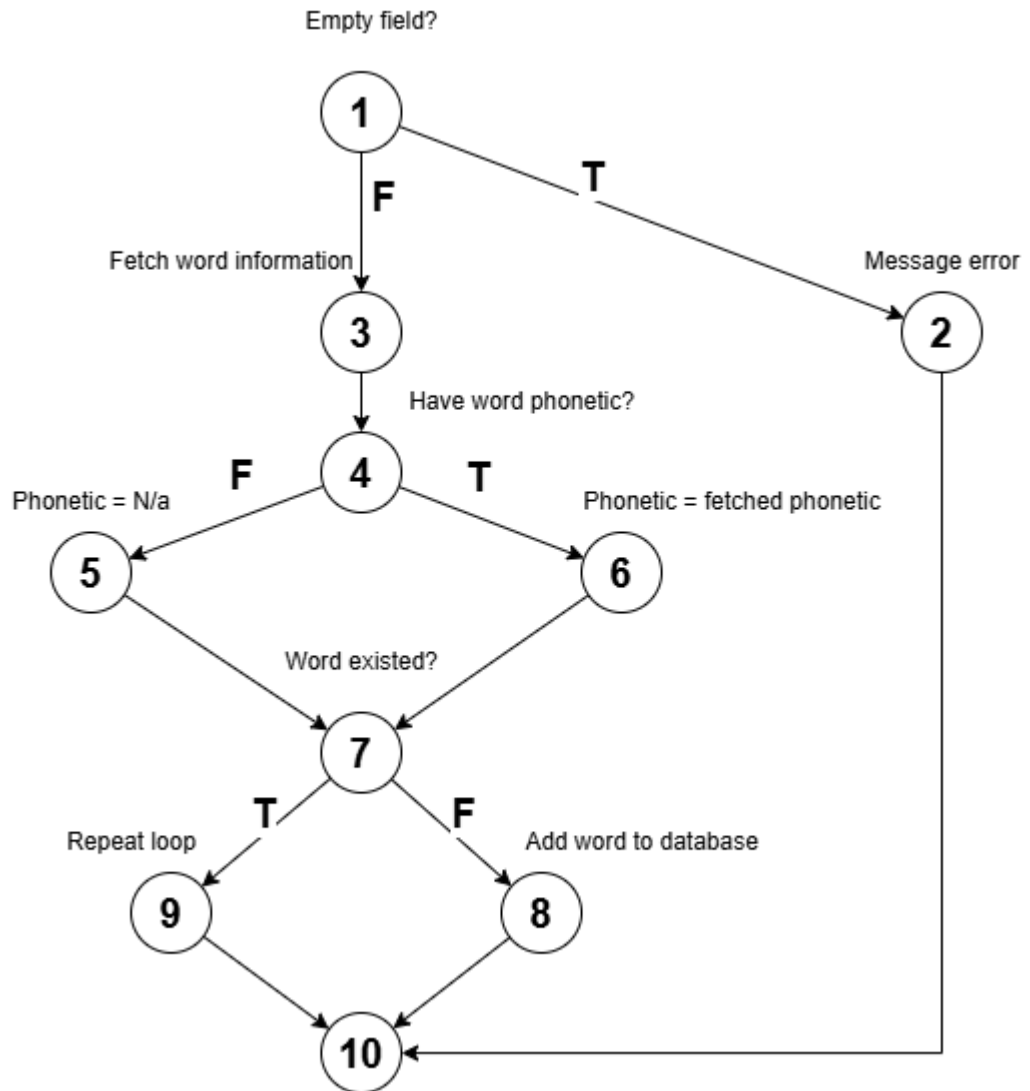


Figure 6 Manual word adding testing flow

TC01: 1 – 2 (empty field = true)

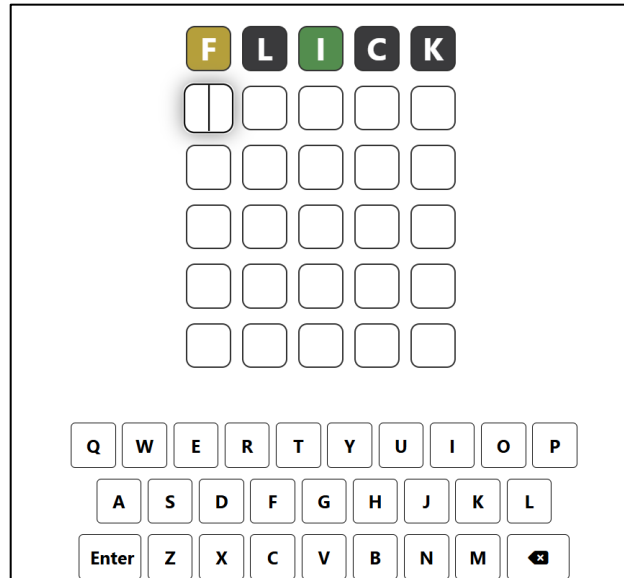
TC02: 1 – 3 – 4 – 5 – 7 – 8 (loop status =true, have no phonetic, word not existed)

TC03: 1 – 3 – 4 – 6 – 7 – 8 (loop status =true, have phonetic, word not existed)

TC04: 1 – 3 – 4 – 6 – 7 – 9 (loop status =true, have phonetic, word existed)

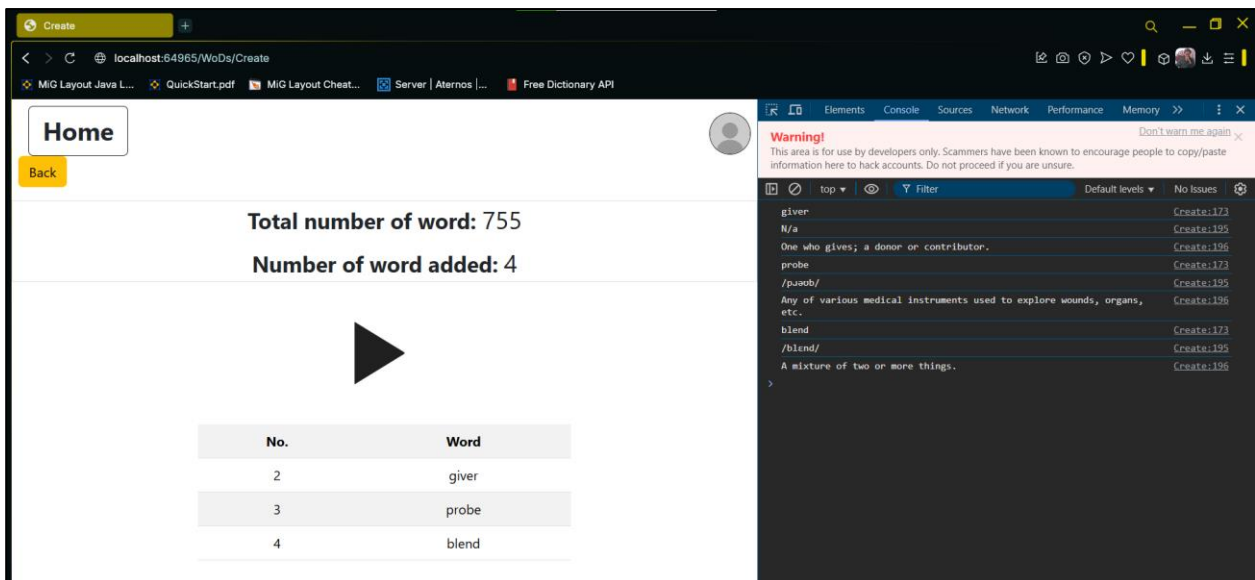
Results and Analysis

1. Unit Testing



- Result: Feedback color return as expected [Yellow, Gray, Green, Gray, Gray].

2. Integration Testing



- Result: Random word API return a word and Dictionary API return the phonetic as well as the word's definition, word with no phonetic have a default value of "N/a".

3. System Testing

Gameplay:

# Test Case	Input Data		Result
	Correct word	Guess left	
TC01	true	3	Game status = win
TC02	false	1	Continue game
TC03	false	0	Game status = loose

Calculate point:

# Test Case	Input Data		Result
	Game status	Difficulty	
TC01	loose	N/a	Point = previous round
TC02	win	Easy	Point = previous + (2+ guess left) * 0.5
TC03	win	Normal	Point = previous + (2+ guess left) * 1
TC04	win	Hard	Point = previous + (2+ guess left) * 3

Add word auto:

# Test Case	Input Data			Result
	Loop status	Have phonetic	Word existed	
TC01	False	N/a	N/a	Cancel adding process
TC02	True	False	False	Add word to database with phonetic = “N/a”
TC03	True	True	False	Add word to database with phonetic = fetched phonetic
TC04	True	N/a	True	Repeat loop

Add word manual:

# Test Case	Input Data			Result
	Empty field	Have phonetic	Word existed	
TC01	True	N/a	N/a	Show message error
TC02	False	False	False	Add word to database with phonetic = “N/a”
TC03	False	True	False	Add word to database with phonetic = fetched phonetic
TC04	False	N/a	True	Show message error

CONCLUSION

The Wordle web-based game project successfully achieved its initial objectives by delivering an interactive and engaging platform that allows users to guess randomly generated five-letter English words. Using the ASP.NET MVC framework, the project implemented key features such as random word generation, user input validation, real-time feedback, and integration with external dictionary APIs for phonetic and definition support.

One of the key outcomes of the project was the seamless user experience with accurate word comparison logic and responsive UI behavior. The system was thoroughly tested and validated through unit, integration, and system testing, ensuring it functioned as intended. The use of modern development practices, such as code-first design and external API consumption, helped streamline the development process and improve project maintainability.

For future work, several enhancements could be considered, such as implementing a ranking system, supporting multiple difficulty levels or languages, improving accessibility for visually impaired users, and developing a mobile-friendly version of the application. Additionally, caching API responses and implementing offline functionality would further improve performance and usability.

Overall, the project successfully demonstrates a practical and entertaining application of web development techniques and contributes positively to vocabulary building in English learners.

REFERENCES

[1] **William H. Press, Saul A. Teukolsky, William T. Vetterling and Brian P. Flannery**, *Numerical Recipes in C, The Art of Scientific Computing* Second Edition, Chapter 7: Random numbers. United States of America, 1992.

[2] **Microsoft Learn**, *Random Class (System)*. Last visited 02/2025.
<https://learn.microsoft.com/en-us/dotnet/api/system.random?view=net-9.0&redirectedfrom=MSDN>

[3] **Wikipedia**, *Subtractive generator*. Last visited 02/2025.
https://rosettacode.org/wiki/Subtractive_generator